

Scientific Calculator with DevOps

CI/CD Pipeline Implementation

Mini Project Report

Software Production Engineering (CS816)



Aayank Singhai

Roll No: **MT2025001**

International Institute of Information Technology, Bangalore

March 2026

Contents

1	Project Overview	2
1.1	Project Objectives	2
1.2	Technology Stack	2
2	Application Architecture	2
2.1	Project Structure	2
2.2	Calculator Features	3
3	Jenkinsfile	3
3.1	Pipeline Configuration	3
3.2	Stage 1: Clone Git Repository	4
3.3	Stage 2: Build Maven Project	4
3.4	Stage 3: Test Maven Project	4
3.5	Stage 4: Build Docker Image	5
3.6	Stage 5: Push to Docker Hub	5
3.7	Stage 6: Deploy with Ansible	5
3.8	Post-Build Actions: Email Notifications	6
4	Pipeline Flow Diagram	7
5	Test Results	8
5.1	Test Cases Overview	8
5.2	Test Execution Screenshot	9
5.3	Test Observations	9
6	Pipeline Execution Screenshots	10
6.1	Successful Pipeline Run	10
6.2	Docker Hub Image	10
6.3	Application Running in Container	11
6.4	Calculator UI Demo	11
6.4.1	UI Features Demonstrated	11
6.4.2	Sample Calculations Demo	12
6.5	Email Notification (Build Failure Build Fixed)	13
7	GitHub Webhook Integration	14
7.1	Webhook Configuration	14
7.2	Jenkins SCM Configuration	14
7.3	Webhook Flow	14
7.4	Webhook Configuration	15
7.5	Automatic Build Trigger	16
8	Ansible Deployment	16
9	Conclusion	17

1 Project Overview

This project implements a **Scientific Calculator** application using Java Swing with a complete DevOps pipeline for automated build, test, and deployment. The project demonstrates the integration of modern CI/CD practices using Jenkins, Docker containerization, and Ansible-based deployment.

1.1 Project Objectives

- Develop a functional scientific calculator with a modern GUI
- Implement comprehensive unit testing using JUnit
- Create an automated CI/CD pipeline using Jenkins
- Containerize the application using Docker with web-based access
- Automate deployment using Ansible
- Implement email notifications for build status

1.2 Technology Stack

Category	Technology	Purpose
Language	Java 11	Application development
GUI Framework	Java Swing	User interface
Build Tool	Apache Maven	Dependency management & build
Testing	JUnit 4.13.2	Unit testing
CI/CD	Jenkins	Pipeline automation
Containerization	Docker + NoVNC	Web-accessible container
Deployment	Ansible	Automated deployment
Version Control	Git/GitHub	Source code management

Table 1: Technology Stack

2 Application Architecture

2.1 Project Structure

The project follows a standard Maven directory structure with clear separation between main source code and test code:

```

1 CS816-SPE-MiniProject-Scientific-Calculator-With-DevOps/
2 |-- src/
3 |   |-- main/java/com/calculator/
4 |   |   |-- Calculator.java           # GUI Application
5 |   |   |-- CalculatorLogic.java      # Business Logic
6 |   |-- test/java/com/calculator/
7 |   |   |-- CalculatorLogicTest.java  # Unit Tests
8 |-- Dockerfile                       # Container configuration
9 |-- Jenkinsfile                      # CI/CD Pipeline

```

```

10 |-- deploy.yml           # Ansible Playbook
11 |-- inventory.ini       # Ansible Inventory
12 |-- pom.xml             # Maven Configuration

```

Listing 1: Project Directory Structure

2.2 Calculator Features

The scientific calculator implements the following operations:

Operation	Symbol	Constraints
Addition	+	None
Subtraction	-	Also handles negative numbers
Multiplication	*	None
Division	/	Divisor $\neq 0$
Square Root	$\sqrt{\quad}$	Input ≥ 0
Factorial	$n!$	Non-negative integer ≤ 170
Natural Logarithm	$\ln$	Input > 0
Power	x^y	Valid mathematical operation

Table 2: Calculator Operations

3 Jenkinsfile

The CI/CD pipeline is defined in the `Jenkinsfile` using declarative pipeline syntax. Below is a detailed explanation of each component.

3.1 Pipeline Configuration

```

1 pipeline {
2   agent any
3   environment {
4     DOCKER_IMAGE_NAME = 'scientific-calculator'
5     GITHUB_REPO_URL = 'https://github.com/aayanksinghai/
6       CS816-SPE-MiniProject-Scientific-Calculator-With-DevOps'
7     DOCKER_HUB_USERNAME = 'aayanksinghai'
8     EMAIL_RECIPIENT = 'aayanksinghai02@gmail.com'
9   }

```

Listing 2: Pipeline Environment Configuration

Explanation:

- `agent any`: Pipeline can run on any available Jenkins agent
- `environment`: Defines global environment variables used across all stages
- Variables store Docker image name, GitHub repository URL, Docker Hub credentials, and email recipient for notifications

3.2 Stage 1: Clone Git Repository

```
1 stage('Clone Git') {
2     steps {
3         script {
4             git branch: 'main',
5                 credentialsId: 'github_credentials',
6                 url: "${GITHUB_REPO_URL}"
7         }
8     }
9 }
```

Listing 3: Git Clone Stage

Purpose: Fetches the latest source code from the GitHub repository.

Key Elements:

- Clones the `main` branch
- Uses stored Jenkins credentials (`github_credentials`) for authentication
- Repository URL is referenced from environment variables

3.3 Stage 2: Build Maven Project

```
1 stage('Build the Maven Project') {
2     steps {
3         sh 'mvn clean package -DskipTests'
4     }
5 }
```

Listing 4: Maven Build Stage

Purpose: Compiles the Java source code and packages it into an executable JAR file.

3.4 Stage 3: Test Maven Project

```
1 stage('Test the Maven project') {
2     steps {
3         sh 'mvn test'
4     }
5 }
```

Listing 5: Maven Test Stage

Purpose: Executes all JUnit test cases to validate application functionality.

Test Execution:

- Runs all tests in `src/test/java/`
- Generates test reports in `target/surefire-reports/`
- Pipeline fails if any test fails

3.5 Stage 4: Build Docker Image

```
1 stage('Build Docker Image') {
2     steps {
3         script {
4             docker.build("${DOCKER_IMAGE_NAME}", '.')
5         }
6     }
7 }
```

Listing 6: Docker Build Stage

Purpose: Creates a Docker image containing the calculator application with NoVNC for web access.

Docker Build Process:

- Uses Dockerfile in project root
- Image includes: OpenJDK 17, Xvfb, x11vnc, NoVNC, Openbox
- Enables GUI application to run in containerized environment

3.6 Stage 5: Push to Docker Hub

```
1 stage('Push Docker Image to Docker Hub') {
2     steps {
3         script {
4             docker.withRegistry('', 'DockerHubCred') {
5                 sh "docker tag ${DOCKER_IMAGE_NAME}
6                     ${DOCKER_HUB_USERNAME}/${DOCKER_IMAGE_NAME}:latest"
7                 sh "docker push
8                     ${DOCKER_HUB_USERNAME}/${DOCKER_IMAGE_NAME}:latest"
9             }
10        }
11    }
12 }
```

Listing 7: Docker Push Stage

Purpose: Pushes the built Docker image to Docker Hub registry for distribution.

Process:

- Authenticates with Docker Hub using stored credentials
- Tags image with Docker Hub username and latest tag
- Pushes image to aayanksinghai/scientific-calculator:latest

3.7 Stage 6: Deploy with Ansible

```
1 stage('Deploy with Ansible') {
2     steps {
3         sh 'ansible-playbook -i inventory deploy.yml'
4     }
5 }
```

Listing 8: Ansible Deployment Stage

Purpose: Automates application deployment using Ansible playbook.

Ansible Playbook Actions:

1. Pulls latest Docker image from Docker Hub
2. Stops and removes existing container (if any)
3. Starts new container with port mapping (8081:8080)
4. Verifies container status

3.8 Post-Build Actions: Email Notifications

```
1 post {
2     failure {
3         script {
4             mail to: "${EMAIL_RECIPIENT}",
5                 subject: "BUILD FAILED: ${env.JOB_NAME} -
6                     Build #${env.BUILD_NUMBER}",
7                 body: ""Build Failed!
8                     Project: ${env.JOB_NAME}
9                     Build Number: ${env.BUILD_NUMBER}
10                    Build URL: ${env.BUILD_URL}
11                    Please check the console output.""
12            }
13        }
14        fixed {
15            script {
16                mail to: "${EMAIL_RECIPIENT}",
17                    subject: "BUILD FIXED: ${env.JOB_NAME} -
18                        Build #${env.BUILD_NUMBER}",
19                    body: ""Build Fixed!
20                        The build is back to normal.""
21            }
22        }
23    }
```

Listing 9: Email Notification Configuration

Purpose: Sends email notifications based on build status.

Notification Triggers:

- **failure:** Sends email when build fails at any stage
- **fixed:** Sends email when a previously failed build succeeds

4 Pipeline Flow Diagram

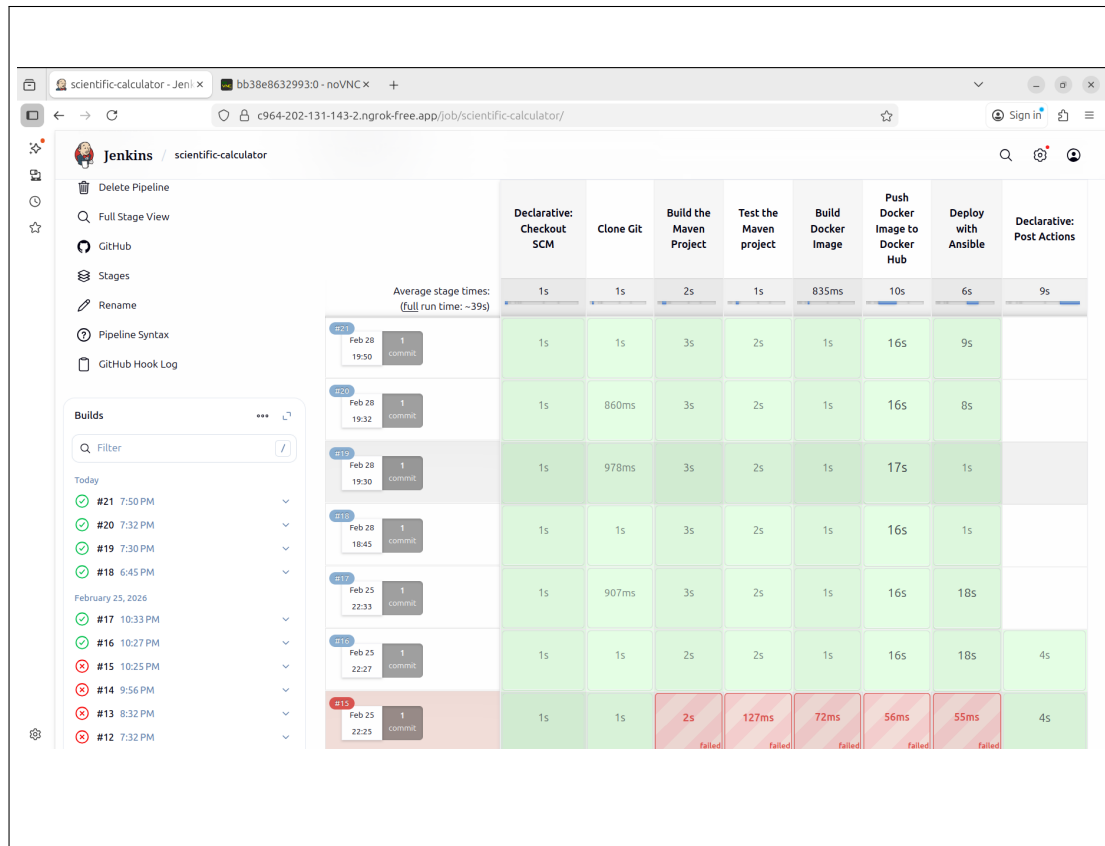
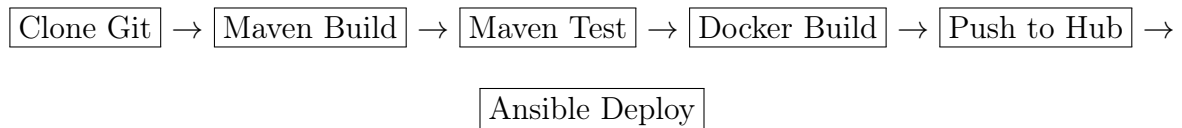


Figure 1: Jenkins Pipeline Stages Visualization

The pipeline executes in the following sequential order:



5 Test Results

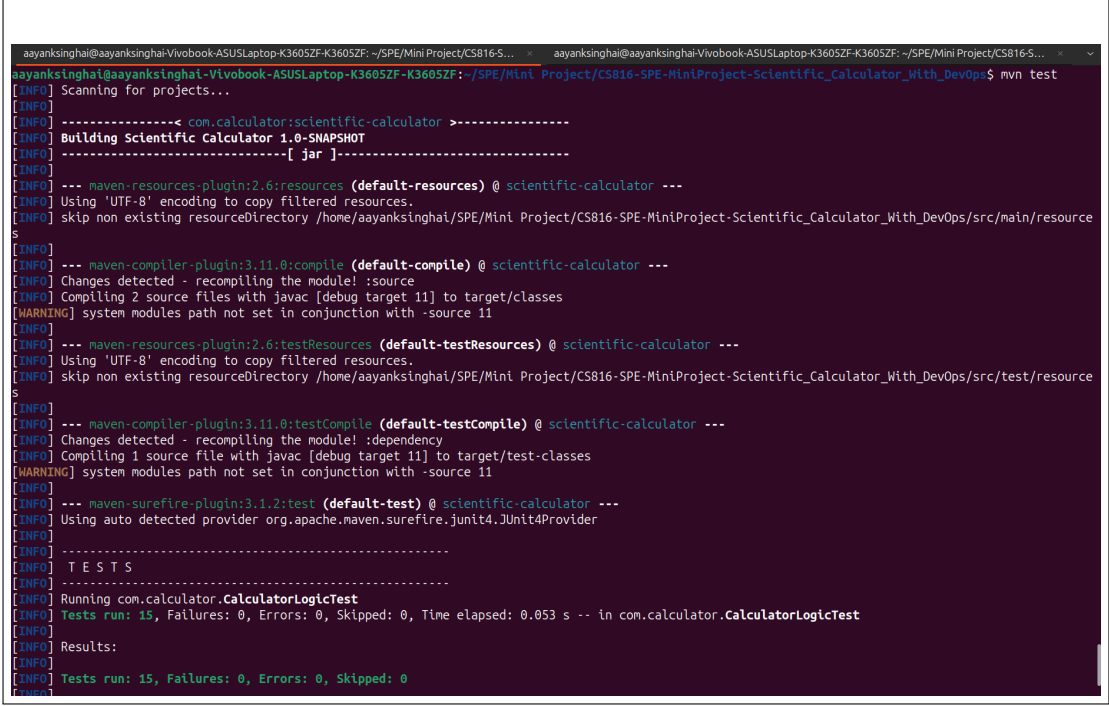
5.1 Test Cases Overview

The test suite `CalculatorLogicTest.java` contains comprehensive unit tests for all calculator operations:

Test Method	Status	Description
testAddition	PASS	Tests addition with positive, negative, and decimal numbers
testSubtraction	PASS	Tests subtraction operations
testMultiplication	PASS	Tests multiplication including zero
testDivision	PASS	Tests division with various inputs
testDivisionByZero	PASS	Verifies <code>ArithmeticException</code> is thrown
testZeroOperations	PASS	Tests operations involving zero
testSquareRoot	PASS	Tests square root of positive numbers
testSquareRootNegative	PASS	Verifies exception for negative input
testFactorial	PASS	Tests factorial (0! to 10!)
testFactorialNegative	PASS	Verifies exception for negative input
testFactorialNonInteger	PASS	Verifies exception for decimal input
testNaturalLog	PASS	Tests $\ln(1)$, $\ln(e)$, $\ln(10)$, $\ln(2)$
testNaturalLogZero	PASS	Verifies exception for zero
testNaturalLogNegative	PASS	Verifies exception for negative input
testPower	PASS	Tests power function with various exponents

Table 3: Unit Test Results Summary

5.2 Test Execution Screenshot



```

aayankingsinghai@aayankingsinghai-Vivobook-ASUSLaptop-K3605ZF-K3605ZF: ~/SPE/Mini Project/CS816-S...
aayankingsinghai@aayankingsinghai-Vivobook-ASUSLaptop-K3605ZF-K3605ZF: ~/SPE/Mini Project/CS816-SPE-MiniProject-Scientific_Calculator_With_DevOps$ mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.calculator:scientific-calculator >-----
[INFO] Building Scientific Calculator 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ scientific-calculator ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /home/aayankingsinghai/SPE/Mini Project/CS816-SPE-MiniProject-Scientific_Calculator_With_DevOps/src/main/resource
[INFO]
[INFO] --- maven-compiler-plugin:3.11.0:compile (default-compile) @ scientific-calculator ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 2 source files with javac [debug target 11] to target/classes
[WARNING] system modules path not set in conjunction with -source 11
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ scientific-calculator ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /home/aayankingsinghai/SPE/Mini Project/CS816-SPE-MiniProject-Scientific_Calculator_With_DevOps/src/test/resource
[INFO]
[INFO] --- maven-compiler-plugin:3.11.0:testCompile (default-testCompile) @ scientific-calculator ---
[INFO] Changes detected - recompiling the module! :dependency
[INFO] Compiling 1 source file with javac [debug target 11] to target/test-classes
[WARNING] system modules path not set in conjunction with -source 11
[INFO]
[INFO] --- maven-surefire-plugin:3.1.2:test (default-test) @ scientific-calculator ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.calculator.CalculatorLogicTest
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in com.calculator.CalculatorLogicTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
[INFO]

```

Figure 2: Maven Test Execution Results

5.3 Test Observations

- **Total Tests:** 15 test methods covering all calculator operations
- **Pass Rate:** 100% (all tests pass successfully)
- **Exception Handling:** 6 tests verify proper exception handling for invalid inputs
- **Edge Cases:** Tests include boundary conditions like factorial of 0, $\ln(1)$, and division by zero
- **Precision:** Floating-point comparisons use delta of 0.001 for accuracy

6 Pipeline Execution Screenshots

6.1 Successful Pipeline Run

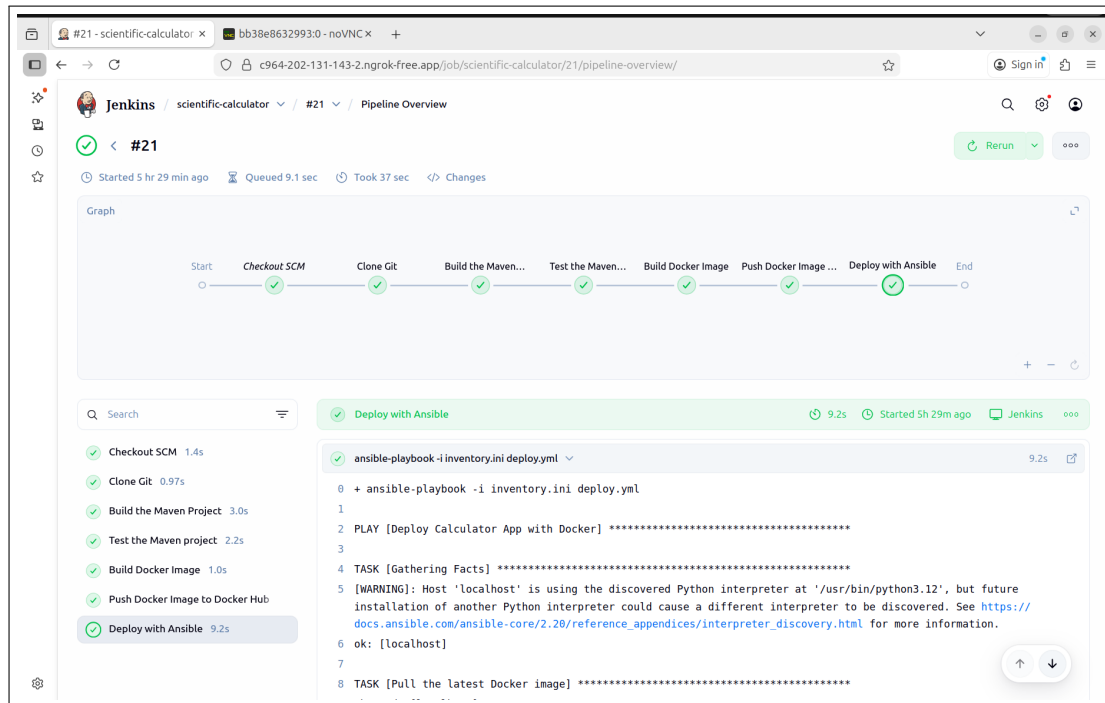


Figure 3: Successful Pipeline Execution

6.2 Docker Hub Image

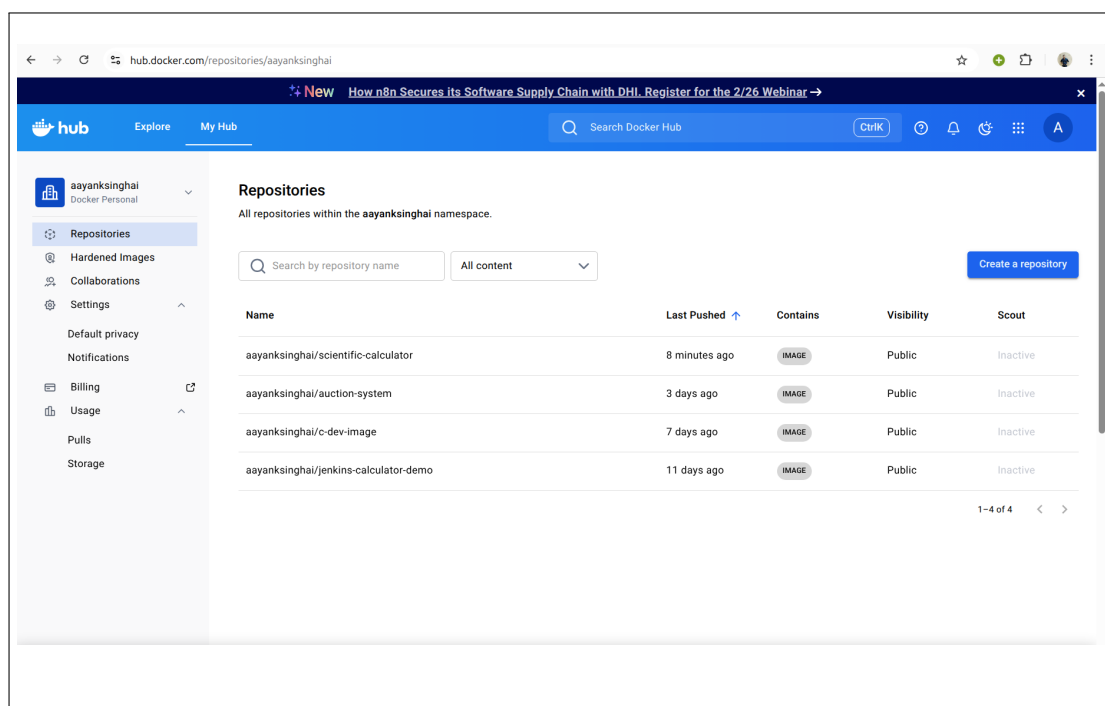
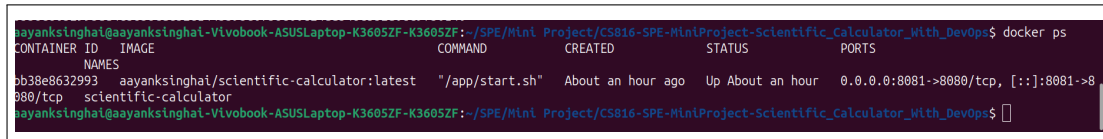


Figure 4: Docker Hub Repository

6.3 Application Running in Container



```

aayanksinghai@aayanksinghai-Vivobook-ASUSLaptop-K3605ZF-K3605ZF:~/SPE/Mini Project/CS816-SPE-MiniProject-Scientific_Calculator_With_DevOps$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
bb38e8632993   aayanksinghai/scientific-calculator:latest  "/app/start.sh"         About an hour ago    Up About an hour    0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
scientific-calculator
aayanksinghai@aayanksinghai-Vivobook-ASUSLaptop-K3605ZF-K3605ZF:~/SPE/Mini Project/CS816-SPE-MiniProject-Scientific_Calculator_With_DevOps$

```

Figure 5: Calculator Application via NoVNC

6.4 Calculator UI Demo

The calculator features a modern dark-themed GUI with color-coded buttons for enhanced user experience:

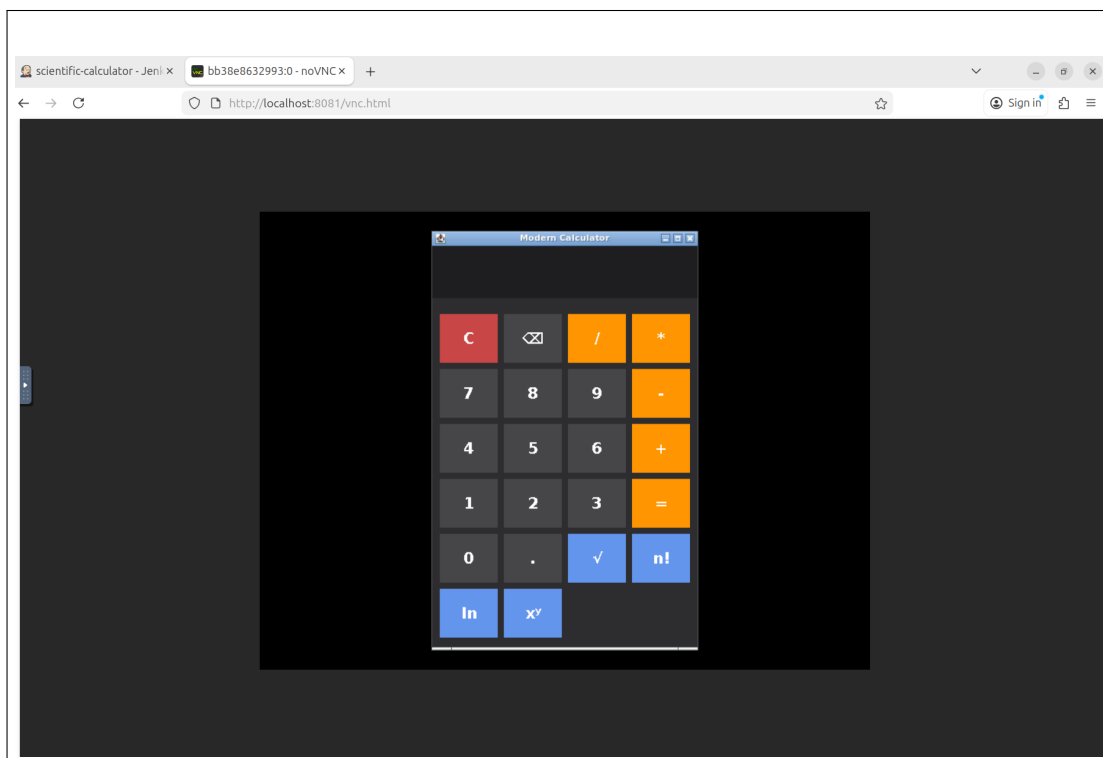


Figure 6: Calculator User Interface

6.4.1 UI Features Demonstrated

- **Expression Display:** Secondary display showing current expression (e.g., "5 + 3 =")
- **Result Display:** Primary display showing input numbers and calculation results

- **Color Scheme:**

- Orange buttons: Arithmetic operators ($+$, $-$, $*$, $/$, $=$)
- Blue buttons: Scientific functions ($\sqrt{}$, $n!$, $\ln$, x^y)
- Red button: Clear (C)
- Gray buttons: Number pad (0-9) and decimal point

- **Error Handling:** Red text display for error messages (e.g., "Can't divide by 0")

6.4.2 Sample Calculations Demo

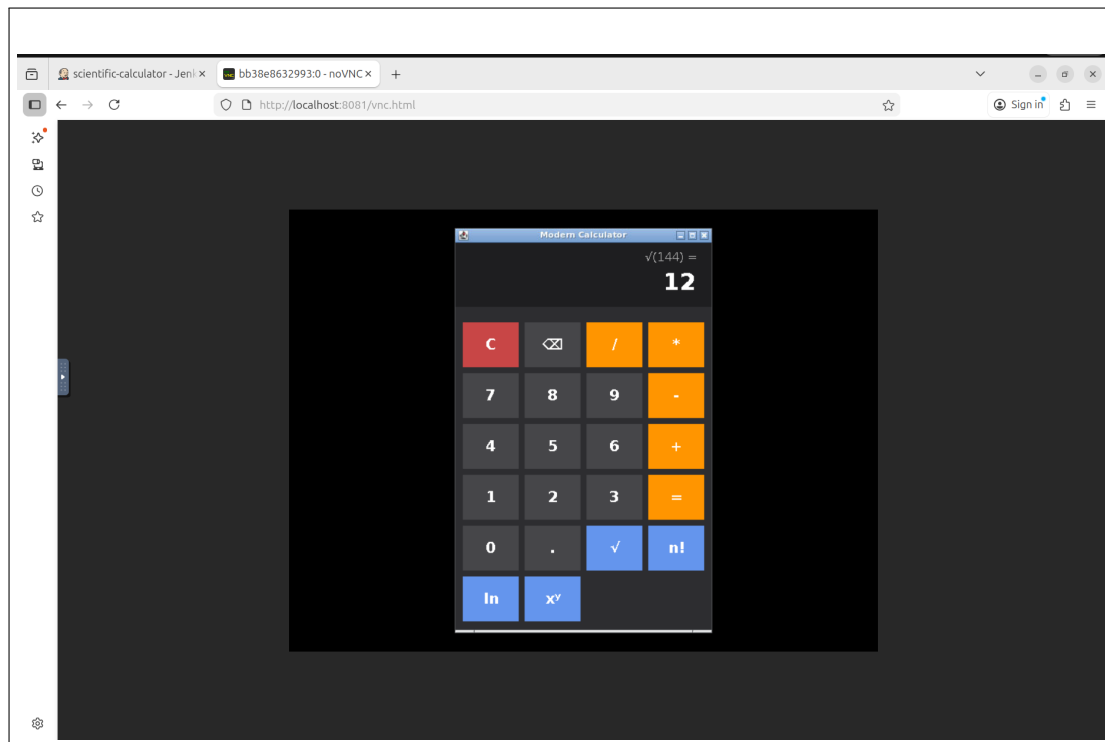


Figure 7: Calculator Operations Demo

6.5 Email Notification (Build Failure Build Fixed)

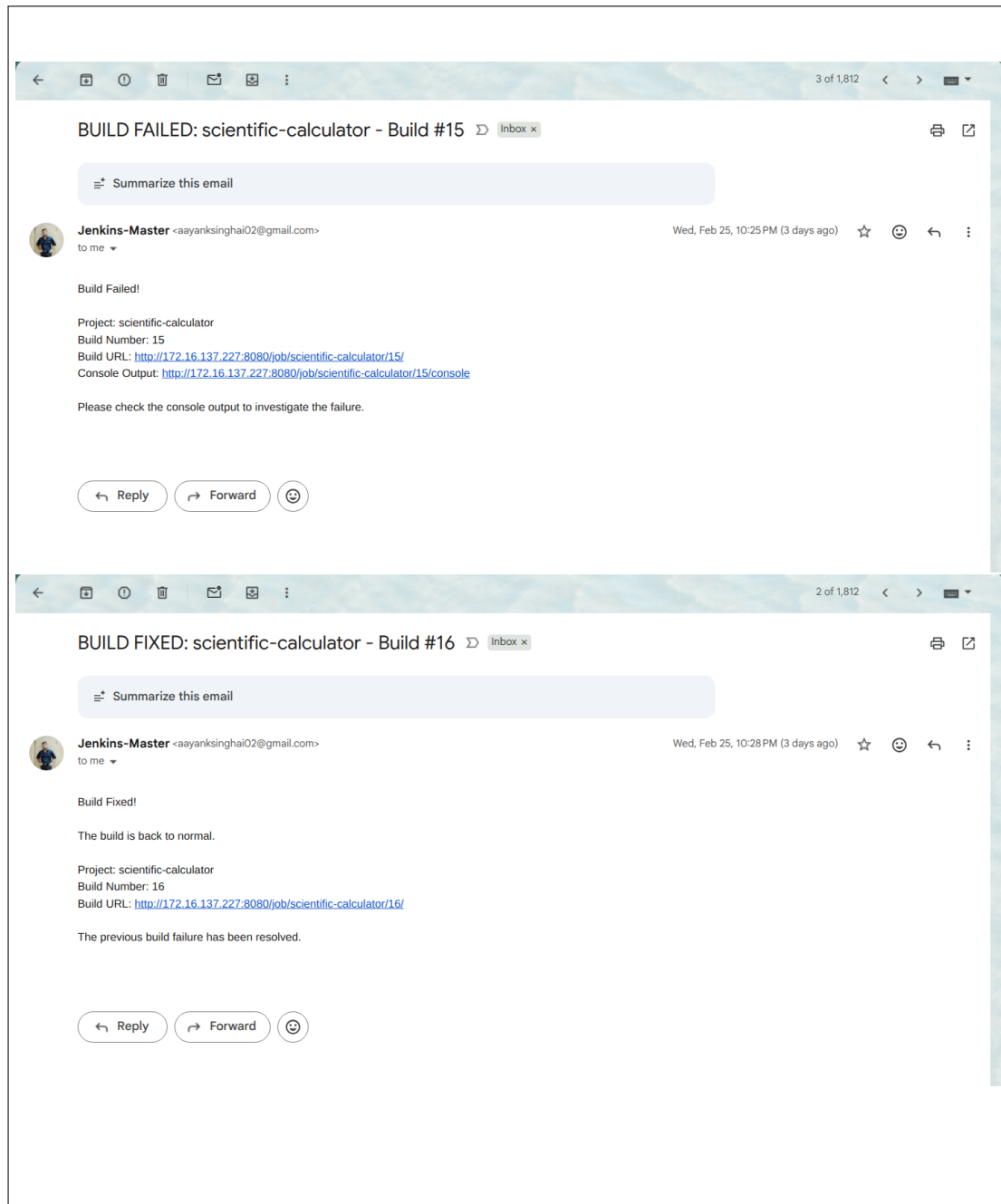


Figure 8: Build Failure Build Fixed Email Notification

7 GitHub Webhook Integration

The project integrates GitHub Webhooks with Jenkins to enable automatic build triggers whenever code is pushed to the repository. This implements a true Continuous Integration workflow where every commit automatically initiates the pipeline.

7.1 Webhook Configuration

The webhook is configured in the GitHub repository settings to notify Jenkins on push events:

- **Payload URL:** `http://<ngrok-url>/github-webhook/`
- **Content Type:** `application/json`
- **Trigger Event:** Push events to the repository
- **Branch Filter:** `main branch`

7.2 Jenkins SCM Configuration

In the Jenkins job configuration, GitHub hook trigger is enabled:

- **Build Trigger:** GitHub hook trigger for GITScm polling
- **SCM:** Git with repository URL and credentials
- **Branch Specifier:** `*/main`

7.3 Webhook Flow

Since Jenkins is running on a local machine without a public IP, **ngrok** is used to create a secure tunnel that exposes the local Jenkins server to the internet, allowing GitHub to send webhook requests.

1. Developer pushes code to GitHub repository
2. GitHub sends POST request to the ngrok public URL (e.g., `https://xxxx-xxxx.ngrok-free.app`)
3. Ngrok receives the request and forwards it through the secure tunnel to the local Jenkins server (`localhost:8080`)
4. Jenkins receives the webhook payload and validates the request
5. Jenkins triggers the pipeline automatically
6. Pipeline executes all stages (Clone, Build, Test, Docker, Deploy)
7. Build status is reported back and email notifications are sent if needed

Ngrok Configuration:

- **Command:** `ngrok http 8080`
- **Public URL:** Dynamically generated (e.g., `https://xxxx-xxxx.ngrok-free.app`)
- **Webhook Payload URL:** `<ngrok-url>/github-webhook/`

Note: The ngrok URL changes each time ngrok is restarted (unless using a paid plan with reserved domains). The GitHub webhook payload URL must be updated accordingly.

```

ayankingsinghai@ayankingsinghai-Vivobook-ASUSLaptop-K3605ZF-K3605ZF: ~/SPE/Mini Project/CS816-SPE-MiniProject-Scientific_Calculator_With_DevOps
ayankingsinghai@ayankingsinghai-Vivobook-ASUSLaptop-K3605ZF-K3605ZF: ~/SPE/Mini Project/CS816-SPE-MiniProject-Scientific_Calculator_With_DevOps
ngrok
F0ne gatwayAfor every AI model.cAvailable in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             ayankingsinghai02@gmail.com (Plan: Free)
Version             3.36.1
Region              India (in)
Latency              16ms
Web Interface        http://127.0.0.1:4040
Forwarding            https://c964-202-131-143-2.ngrok-free.app -> http://localhost:8080

Connections
  ttl    opn    rt1    rt5    p50    p90
  361    0      0.00   0.01   0.04   30.87

HTTP Requests
-----
01:25:07.660 IST GET /job/scientific-calculator/20/statusIcon 20
01:25:02.603 IST GET /adjuncts/d28b7f98/lib/layout/copyButton/copyButton.js 20
01:25:02.605 IST GET /static/d28b7f98/descriptor/hudson.tasks_ant.AntOutcomeNote/style.css 20
01:25:02.618 IST GET /static/d28b7f98/descriptor/hudson.console.ExpandableDetailsNote/style.css 20
01:25:02.708 IST GET /l18n/resourceBundle 20
01:25:02.618 IST GET /static/d28b7f98/descriptor/hudson.plugins.gradle.GradleTaskNote/script.js 20
01:25:02.618 IST GET /static/d28b7f98/descriptor/hudson.plugins.gradle.GradleTaskNote/style.css 20
01:25:02.603 IST GET /static/d28b7f98/hudson.console.ConsoleAnnotatorFactory/hudson.plugins.timestampator.TimestampAnnotatorFactory3/script.js 20
01:25:02.618 IST GET /static/d28b7f98/descriptor/org.jenkinsci.plugins.workflow.job.console.NewNodeConsoleNote/script.js 20
01:25:02.390 IST GET /job/scientific-calculator/20/console 20
  
```

Figure 9: Ngrok Configuration

7.4 Webhook Configuration

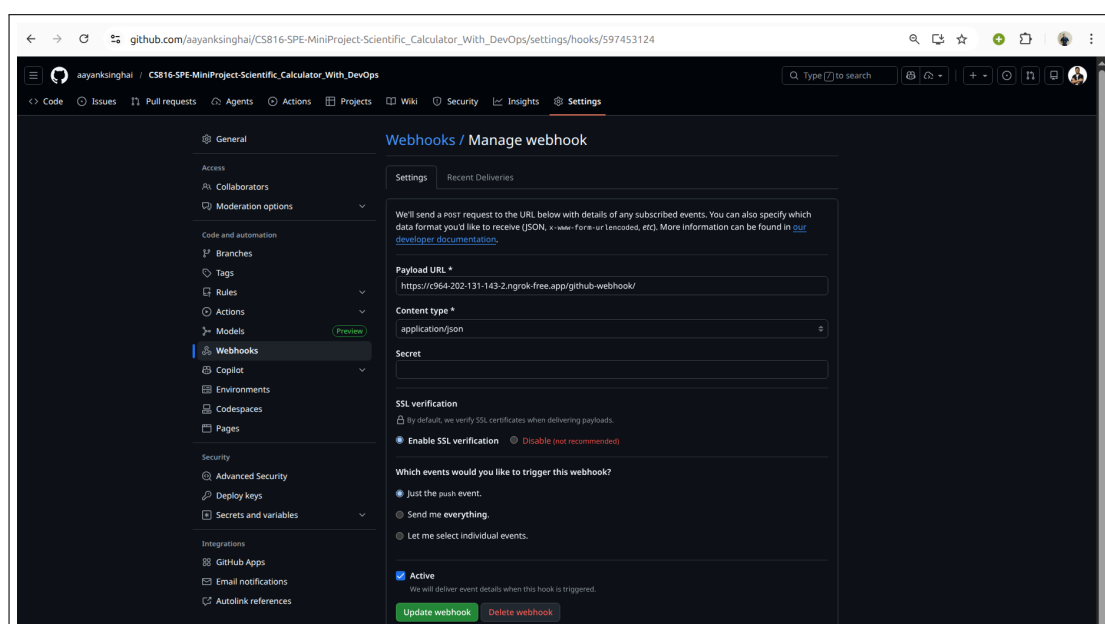


Figure 10: GitHub Webhook Configuration

7.5 Automatic Build Trigger

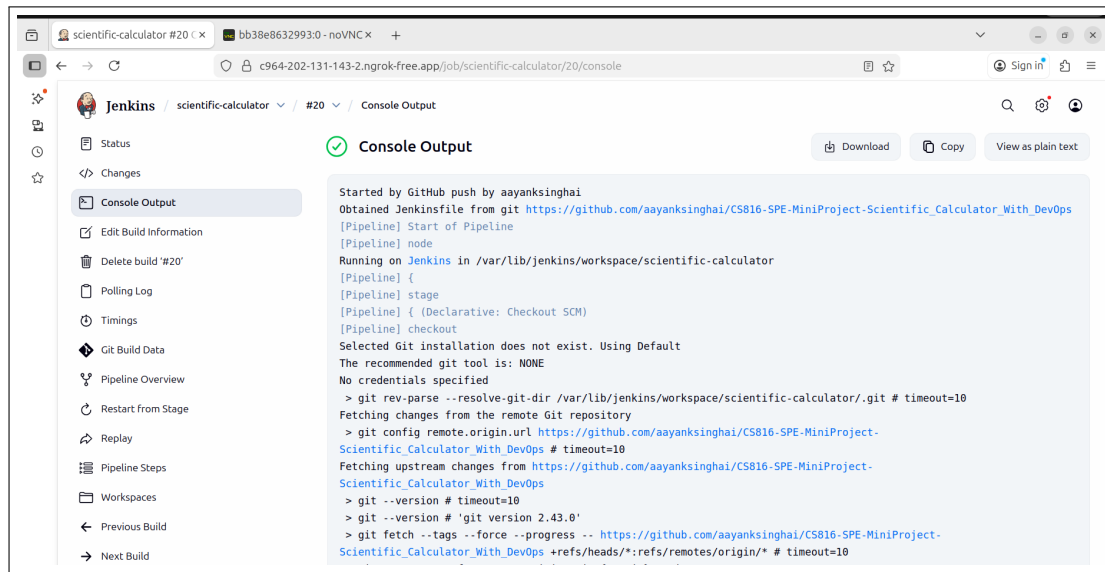


Figure 11: Jenkins Build Triggered by GitHub Webhook

8 Ansible Deployment

The Ansible playbook (deploy.yml) automates the deployment process:

```

1 ---
2 - name: Deploy Calculator App with Docker
3   hosts: webserver
4   tasks:
5     - name: Pull the latest Docker image
6       shell: docker pull aayanksinghai/scientific-calculator:latest
7
8     - name: Stop and remove existing container
9       shell: docker rm -f scientific-calculator
10      ignore_errors: true
11
12     - name: Run calculator container (detached)
13       shell: docker run -d -p 8081:8080 --name scientific-calculator
14              --restart unless-stopped
15              aayanksinghai/scientific-calculator:latest
16
17     - name: Display container status
18       shell: docker ps -f "name=scientific-calculator"

```

Listing 10: Ansible Playbook (deploy.yml)

Deployment Process:

1. Pulls the latest image from Docker Hub
2. Removes any existing container to avoid conflicts
3. Starts a new container with:
 - Port mapping: Host 8081 → Container 8080
 - Auto-restart policy: **unless-stopped**
 - Detached mode for background execution
4. Verifies deployment by checking container status

9 Conclusion

This project successfully demonstrates the implementation of a complete DevOps pipeline for a Java-based scientific calculator application. Key achievements include:

- **Automated CI/CD Pipeline:** 6-stage Jenkins pipeline covering build, test, containerization, and deployment
- **Comprehensive Testing:** 15 unit tests with 100% pass rate covering all calculator operations and edge cases
- **Docker Containerization:** Web-accessible GUI application using NoVNC technology
- **Automated Deployment:** Ansible-based deployment ensuring consistent and repeatable deployments
- **Notification:** Email notifications for build failures and recovery

The project showcases practical application of DevOps principles including:

- Continuous Integration (automated builds and tests)
- Continuous Deployment (automated container deployment)
- Infrastructure as Code (Jenkinsfile, Dockerfile, Ansible playbook)
- Version Control (Git-based workflow)

GitHub Repository: https://github.com/aayanksinghai/CS816-SPE-MiniProject-Scientific_Calculator_With_DevOps

Docker Hub: <https://hub.docker.com/r/aayanksinghai/scientific-calculator>